CAFE SYSTEM

Java Object-Oriented Programming Project

Kenneth E. Daanoy
Jola Kabristante

Date: November 9, 2026

1. Introduction

The Cafe System is a Java application that demonstrates Object-Oriented Programming (OOP) concepts through a simple cafe ordering scenario. It models customers and three kinds of menu items (beverages, pastries and meals), and lets an order be built from those items, priced, discounted and printed as a receipt.

The program is a console application with no external libraries and no IDE-specific dependencies. It compiles with `javac` and runs with `java CafeApp`.

2. Objectives of the Project

- To build a simple cafe ordering system using Java.
- To apply inheritance, abstraction, polymorphism and encapsulation in a practical program.
- To use constructors and method overloading to give objects flexible ways of being created and used.
- To validate data inside setters so objects cannot be left in an invalid state.
- To compute prices, discounts, vouchers and loyalty points and present them as a formatted receipt.

3. System Features

Feature	Description
Customer records	A customer has a name, a student flag and a loyalty-point balance. A customer created without a name is a "Guest".
Three menu item types	**Beverage** (size S, M or L), **Pastry** (optionally warmed) and **Meal** (optionally upsized, with extra sides).
Order building	An order belongs to one customer and holds up to three items. Adding a fourth item is refused with a message.
Automatic pricing	Each item computes its own price from its base price plus its own fees (see Section 5).
Student discount	Student customers receive a 10% discount on the subtotal.
Vouchers	A peso voucher can be applied to an order. The total never goes below zero.
Loyalty points	Customers earn 1 point for every 50 pesos of the final total. Points are awarded once per order.
Feature	**Description**
Receipt printing	Prints the customer, each item with a dotted leader and aligned price, subtotal, discounts, total and loyalty points.

4. Classes Used in the System

The project has seven Java files: one abstract base class, three subclasses, two supporting classes and the main program.

Class	Role
MenuItem (abstract)	Base class for everything on the menu. Stores name, base price and item code, validates them in setters, and declares the abstract method `computePrice()`.
Beverage	Extends MenuItem. Adds a size (S, M or L). Medium adds a 15.00 fee and Large adds a 25.00 fee.
Pastry	Extends MenuItem. Adds a warmed flag. Warming adds a flat 10.00 fee.
Meal	Extends MenuItem. Adds an upsized flag and a count of extra sides. Upsizing adds 20.00 and each extra side adds 12.00.
Customer	Stores name, student status and loyalty points. Provides overloaded `addPoints` methods.
Order	Holds the customer and up to three MenuItem objects. Computes subtotal, discount and total, and prints the receipt.
CafeApp	Contains `main`. Creates customers, items and two sample orders to demonstrate the system.

Class hierarchy:

```
MenuItem (abstract)
  |-- Beverage
  |-- Pastry
  |-- Meal

Order  --has-a-->  Customer
Order  --has-a-->  MenuItem (up to 3)
```

5. Object-Oriented Programming Concepts Applied

1. Encapsulation

All attributes are declared `private` (or `protected` in MenuItem so subclasses can read the price and name) and are changed only through setters. The setters validate their input. For example, `setBasePrice` rejects negative values, `setName` rejects blank names, and `setSize` accepts only S, M or L.

2. Inheritance

Beverage, Pastry and Meal each extend MenuItem with the `extends` keyword. They reuse the name, base price and item code logic from the parent and add only what is specific to them.

3. Abstraction

MenuItem is an `abstract` class. It declares `computePrice()` without a body, so a generic MenuItem cannot be created and every subclass is forced to define its own pricing rule.

4. Polymorphism (method overriding)

Order stores its items as `MenuItem` references and calls `computePrice()` and `getDescription()` on them. Java runs the version that belongs to the real object (Beverage, Pastry or Meal), so Order never needs to know which kind of item it holds.

5. Method overloading

Class	Overloaded method	Versions
Customer	addPoints	`addPoints(int points)` adds points directly. `addPoints(double amountSpent)` converts pesos to points.
Order	computeTotal	`computeTotal()` and `computeTotal(double voucher)`.
Order	printReceipt	`printReceipt()` and `printReceipt(double voucher)`.
Customer, Beverage, Pastry, Meal	Constructors	Default constructor and parameterized constructors.

6. Constructors

Every class has a default constructor that sets safe values (for example `"Guest"`, `"Unnamed Item"`, size S) and at least one parameterized constructor. `Customer(String name)` uses constructor chaining with `this()`, and the subclasses call `super()`.

7. Constants and the `static final` keyword

Fees and rates are defined once as named constants, such as `STUDENT_DISCOUNT_RATE`, `PESOS_PER_POINT`, `WARMING_FEE` and `LARGE_FEE`, instead of being repeated as unexplained numbers in the code.

6. Program Flow

When `java CafeApp` is run, the program does the following:

1. Creates two customers: Ana Reyes (a student) and Ben Cruz (not a student).
2. Creates menu items for Order 1001 using parameterized constructors (Iced Latte, Ensaymada, Chicken Adobo), plus a Cookie created with the default constructor and filled in with setters.
3. Creates Order 1001 for Ana and adds four items. The order accepts three and rejects the fourth (the Cookie) with the message "Order is full".
4. Prints the receipt for Order 1001: subtotal, 10% student discount, total and loyalty points.
5. Creates a Hot Chocolate and a Beef Tapa with default constructors and setters. A negative base price (-5.0) is deliberately tried and rejected by validation.
6. Creates Order 1002 for Ben and prints its receipt with a 20.00 voucher applied.

7. Sample Output

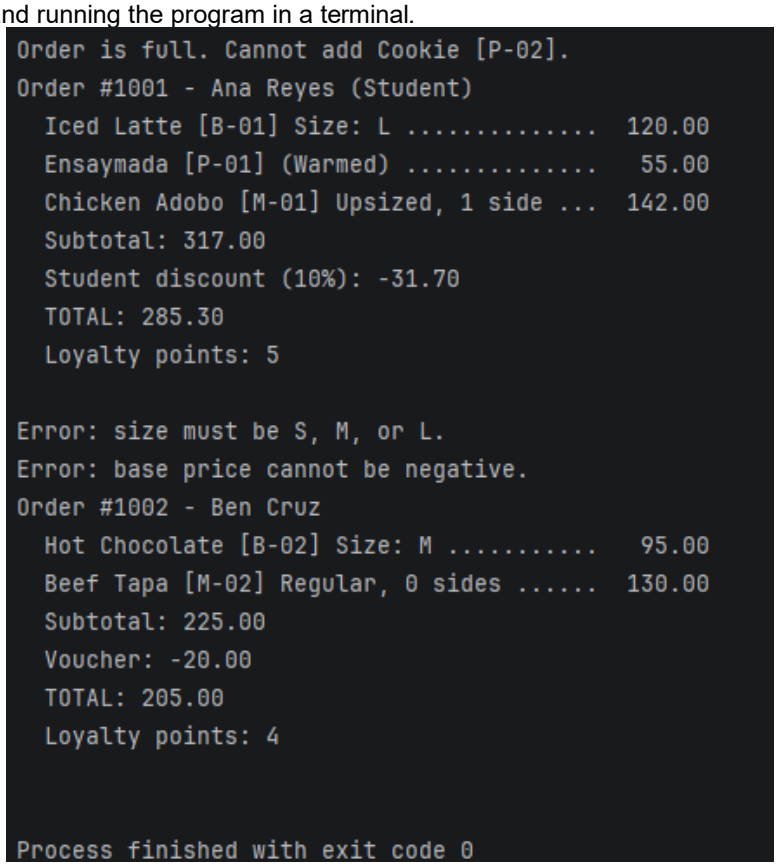
The following is the actual output of the program:

```
Order is full. Cannot add Cookie [P-02].
Order #1001 - Ana Reyes (Student)
```

```
  Iced Latte [B-01] Size: L ..............  120.00
  Ensaymada [P-01] (Warmed) ..............   55.00
  Chicken Adobo [M-01] Upsized, 1 side ...  142.00
  Subtotal: 317.00
  Student discount (10%): -31.70
  TOTAL: 285.30
  Loyalty points: 5

Error: base price cannot be negative.
Order #1002 - Ben Cruz
  Hot Chocolate [B-02] Size: M.............95.00
  Beef Tapa [M-02] Regular, 0 sides .........130.00
  Subtotal: 225.00
  Voucher: -20.00
  TOTAL: 205.00
  Loyalty points: 4
```

Figure 1. Compiling and running the program in a terminal.



Checking the numbers: the Iced Latte costs 95 + 25 (Large) = 120. The warmed Ensaymada costs 45 + 10 = 55. The upsized Chicken Adobo with one extra side costs 110 + 20 + 12 = 142. The subtotal is 317, and Ana's 10% student discount is 31.70, giving 285.30. That earns `(int)(285.30 / 50)` = 5 points. For Ben, the Hot Chocolate is 80 + 15 (Medium) = 95, the subtotal is 225, and after the 20.00 voucher the total is 205, which earns 4 points.

8. How to Compile and Run

Open a terminal in the `Daanoy_CafeSystem` folder (the JDK must be installed) and run:

```
javac *.java
java CafeApp
```

No IDE, build tool or external library is needed.

Documentation

Every class and every public method has a Javadoc comment, and the code has inline comments where the logic needs explaining. The Javadoc web pages can be generated from the project folder with:

```
javadoc -d docs *.java
```

9. Limitations and Possible Improvements

- An Order holds a maximum of three items, stored in three separate fields and limited by the constant `MAX_ITEMS`. Allowing more items would mean adding more item fields.
- Loyalty points are awarded the first time a receipt is printed, so printing is not purely a display action. Moving this into a separate method such as `checkout()` would be cleaner.
- Validation errors are printed to the console rather than thrown as exceptions.
- The program runs a fixed demonstration. A menu-driven interface with user input could be added.

10. Conclusion

Developing the Cafe System showed how Java and OOP concepts can be applied to a simple real-world situation. Through this project I gained a better understanding of how inheritance, abstraction, polymorphism, encapsulation, constructors and method overloading work together in one program. Building the abstract MenuItem class and letting each subclass define its own price made it clear why polymorphism keeps the Order class simple.

The project also improved my logical thinking and problem-solving skills, especially in validating data, working out pricing rules and tracing the results by hand. Overall, this activity provided valuable experience in developing a basic system using Java.